COS 214 Prac 6

COS-214-Prac-6/docs at main · elme21/COS-214-Prac-6

Task 1

Workflow is the automation of a process in which documents and information are passed between participants in accordance with a set of rules. (Hollingsworth, 1995) A workflow management system (WfMS) is software that defines workflows and creates and manages their execution.

A running workflow instance is one particular execution of a workflow. It has its own identity, data, and history, and many instances can run at the same time from the same workflow definition. A workflow definition is a reusable blueprint, with steps, orders, rules, and transitions from which a running workflow instance is built. Activities and workflow items are represented as runtime objects or records inside a workflow instance. (IBM, 2024)

Work items are concrete executions of tasks that are assigned to specific people, groups, or services and routed between workflow steps by the engine according to rules defined in the workflow. Approvals, rejections, and escalations are handled as explicit decision points with timeouts and alternate paths. The workflow pauses at an approval step, waits for a decision or a timeout, then follows the branch that matches the outcome. A rejection prevents continuation and usually triggers an alternate path, and then an escalation handles situations where an approval or work item is not acted on in time. (Temporal, 2026)

Participants are entities that can perform work in a workflow, such as users, groups, or organisational units, and roles are descriptions of the capabilities or responsibilities required to perform certain activities. A user can have multiple roles, and a role can be performed by multiple users. Responsibilities are the tasks or activities to be done by the users. (Hollingsworth, 1994)

Dependencies are modelled as explicit prerequisites that must be satisfied before an activity can run. Events are structured triggers that can start, advance or terminate workflow instances and failures are handled through configurable policies. (Aalst, 1998) Workflow systems' interactions with external systems are modelled as service tasks or external activities that can invoke connectors or adapters implementing a common interface. (Ferreira, 2004)

Workflow systems record history and audit information by logging structured events for every important action and state change in a workflow instance and then storing these events in a log or history tables. (List B., 2001) Running workflows are monitored and reported on by collecting execution data from the workflow engine, exposing it through dashboards, reports and alerts, and using that information to track progress, detect bottlenecks and measure performance against targets. (Dumas, 2018)

A workflow engine is reusable across different processes when it provides a generic execution core and exposes process-specific logic through external workflow definitions so that new processes can be added or modified without changing the engine itself. (Mönckemeyer, 2024)

A workflow definition includes identity and metadata, structure of the process, event and timer definitions, variable and data schema, roles and participating mappings, assignment and routing policies, error handling and escalation configuration. Whereas a running instance contains identity and linkage, current execution state, workflow variables, work items, history and audit trail, timers and pending events, and error and exception state, which are linked to but independent of its definition. (Oracle, 2026)

In real workflow systems, nested activities or sub-processes are modelled as activities that encapsulate an internal workflow, either embedded within the parent definition or defined separately and invoked via a call activity. The engine itself stays generic, while process-specific decisions are externalised into workflow definitions, rules and configuration. The engine provides the mechanisms to evaluate and enforce those decisions, but not the decisions themselves. (OMG, 2010)

In workflow systems, changes to workflow definitions are handled by versioning the definition and binding each running instance to the version it started with, so that edits create a new version used only by new instances, while in-flight instances continue on their original version. (Lotz, 2026)

Bibliography

IBM, 2024. *Workflow definitions and workflow instances.* [Online]
Available at: https://www.ibm.com/docs/en/openpages/8.3.0?topic=fundamentals-workflow-definitions-workflow-instances
[Accessed 4 October 2026].

Temporal, 2026. *Approval Pattern.* [Online]
Available at: https://docs.temporal.io/design-patterns/approval
[Accessed 4 October 2026].

Hollingsworth, D., 1994. *Workflow Management Coalition The Workflow Reference Model.* [Online]
Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html
[Accessed 4 October 2026].

Ferreira, D., 2004. *Developing a Reusable Workflow Engine.* [Online]
Available at: https://web.ist.utl.pt/diogo.ferreira/papers/ferreira04developing.pdf
[Accessed 4 October 2026].

Mönckemeyer, F., 2024. *Hitchhikers Guide to Workflow Engines.* [Online]
Available at: https://devblogs.microsoft.com/ise/guide-to-workflow-engines/
[Accessed 5 October 2026].

Oracle, 2026. *About Workflow Definitions.* [Online]
Available at: https://docs.oracle.com/en/database/oracle/apex/24.2/htmdb/about-workflow-definitions.html#GUID-A72F0427-0228-4F34-9996-D8C4E5D7F0D0
[Accessed 5 October 2026].

Lotz, N., 2026. *Workflow Versioning and Backward Compatibility: Stop Breaking Long-Running Executions.* [Online]
Available at: https://orkes.io/blog/workflow-versioning-and-backward-compatibility-in-conductor
[Accessed 5 October 2026].

Hollingsworth, D., 1995. *Management Coalition The Workflow Reference Model.* [Online]
Available at: https://pozzi.faculty.polimi.it/ImpEl/tc003v11.pdf
[Accessed 5 October 2026].

List B., S. J. B. R., 2001. *Measuring Knowledge with Workflow Management Systems.* [Online]
Available at: https://www.ifs.tuwien.ac.at/~bruckner/pubs/takma2001_knowledge.pdf
[Accessed 4 October 2026].

Aalst, W. v. d., 1998. *The Application of Petri Nets to Workflow Management.* [Online]
Available at: https://www.researchgate.net/publication/220337578_The_Application_of_Petri_Nets_to_Workflow_Management
[Accessed 5 October 2026].

Dumas, M., 2018. *Fundamentals of Business Process Management.* 2nd Edition ed. Berlin: Springer.

OMG, 2010. *Business Process Model and Notation (BPMN).* [Online]
Available at: https://www.omg.org/spec/BPMN/2.0/PDF
[Accessed 5 October 2026].

Task 2

The organisation or environment

TIVIDY is a university enrolment system, aimed at a mid-sized university in South Africa which will provide the application processing of new students and the registration of existing , returning students for a new year. There can be numerous concurrent cases, which is why workflow is important.

The process being managed;

The process of submitting results (either final school/matric results or the tertiary results obtained the previous year), comparing those results with the requirement for the relevant program/course and then accepting/ rejecting the application/registration of the prospective or returning student. Other factors that will also be taken into consideration is the person's personal details, e.g. nationality (verifying citizenship), age (that a realistic age is entered) etc

The participants and roles involved;

Participant	Role
Student (Prospective/Returning)	This is the participant who submits the appropriate documentation and details for the enrolment process.
Verification system	This participant decides whether the enrolment and application has been successful or not, based on comparing the data entered to certain criteria.
Reviewing staff	Admissions officers, finance officers and Registrar. Admission officers will review the applications, while finance officers will review the financial side thereof, including the cost of the course, how the student will pay, payment plan and who will pay (student, parents, bursary loan etc.) and finally if an issue is raised for escalation, the final step will be an escalation to the Registrar, which will deliver a final verdict on registration.

The major pieces of work;

Stage 1: Application and verification	Submit application documents, validate personal details (ID, age, citizenship etc.), Verify results
Stage 2: Admission decision	Compare results to course requirements, faculty review, approval/declining from faculty, applicant accepts/declines
Stage 3: Registration (sub-process)	Pay fees, accept modules, update student record (with modules, admission etc.).

Important decisions and routing points

Some important decisions include:

- Comparing a local vs international vs an alternate local curriculum. In this case different requirements will apply
- Does the applicant meet the minimum requirements for the course? Yes/No
- Is the Course full? If so, place the applicant on a waitlist, or reject the application, or make an offer to the applicant
- Did the returning student pass enough credits to progress? Yes/ No?
- Are the documents complete and submitted? Proceed ahead or request resubmission.

Approvals and dependencies;

Faculty needs a decision. Borderline Cases (e.g. borderline marks for admission) might go to a second level; be escalated (to an HOD or the Registrar).

If a reviewer doesn't respond within a deadline, escalate

Dependecies:
An offer can't be issued until both verification checks pass. Registration can't start until the offer is accepted and module selection can't be confirmed until fees are paid.(or arrangement is approved)

Possible failures or exceptions;

International students or students following an alternate curriculum (e.g. the Cambridge curriculum in South Africa) will not be able to be measured by the same criteria local students will be measured by.

Unreadable documents, applicants not responding by the deadline, payment failures or delays, Reviewer unresponsive (requires escalation) or a course reaching capacity can lead to failures.

External systems where relevant

External systems will include systems in which the validity of, for example, an ID number can be cross checked, a results database where a candidate number is submitted and results returned, Payment gateway etc.

The expected outcome of the workflow

The expected outcome will be that a student can successfully enrol or re-enrol for a University course, Rejected, waitlisted, application expired due to no response or an application being withdrawn by an applicant.

Another example of a specific scenario is healthcare admission:

The organisation or environment;

TIVIDY is a university enrolment system, aimed at a mid-sized hospital in South Africa which will provide the patient's admission to discharge. Many patients are processed at once, so each admission is its own workflow.

The process being managed;

Registering the patient, verifying identity, deciding whether to admit, allocating a bed, treatment orders, escalation to other medical staff/specilists.

• The participants and roles involved;

Participant	Role
Patient (new/Returning)	This is the participant who submits the appropriate documentation and symptoms/details for the admission process.
Verification system	This participant decides whether the admission has been successful or not, based on comparing the data entered to certain criteria. Admitting doctor does clinical assessment and decides on escalation to specialists
Reviewing staff	Admissions officers, medical staff, HOD, billing officer

• The major pieces of work;

Stage 1: Registration and verification	Submit admission documents, validate personal details (ID, age, citizenship etc.), Verify results
Stage 2: Clinical assessment and Admission decision	Emergency route, Compare results to admission or specialist requirements, approval/declining sending to specialist, patient accepts/declines.
Stage 3: \Treatment and discharge (sub-process)	Add to ward, treatment orders, pay fees, accept treatment, update medical record (with treatment, admission etc.).

• important decisions and routing points

Some important decisions include:

- Emergency or elective Route
- Funding type – medical aid, or self-funded
- Ward or high care
- Are documents complete? Proceed or request resubmission
- Is patient stable for discharge

• approvals and dependencies;

Specialist must approve high cost procedures and borderline cases.

If doctor doesn't respond within deadline, can escalate to HOD.

Dependencies:
Assessment order can't proceed until assessment is complete. With the elective route, a bed cannot be allocated until pre-authorisation is complete or deposit/other financial arrangement is in place.

• possible failures or exceptions;

No bed available

No doctor/specialist available

Payment failure

Delayed lab results

• external systems where relevant

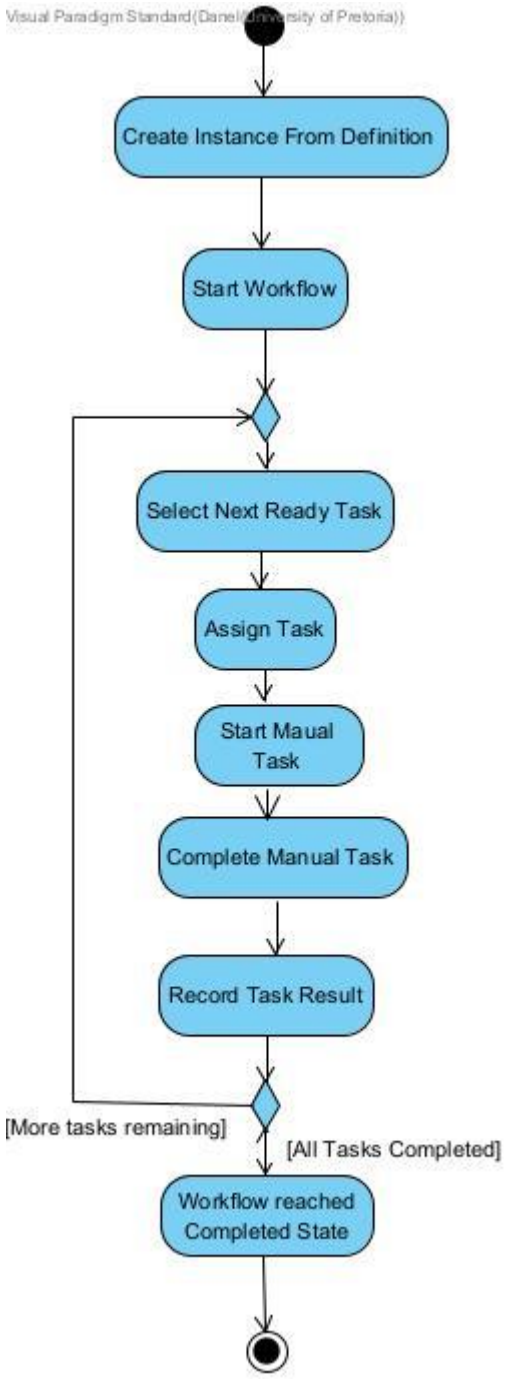
External systems will include systems in which the validity of for example an ID number can be cross checked, medical aid services, external labs for testing.

• the expected outcome of the workflow

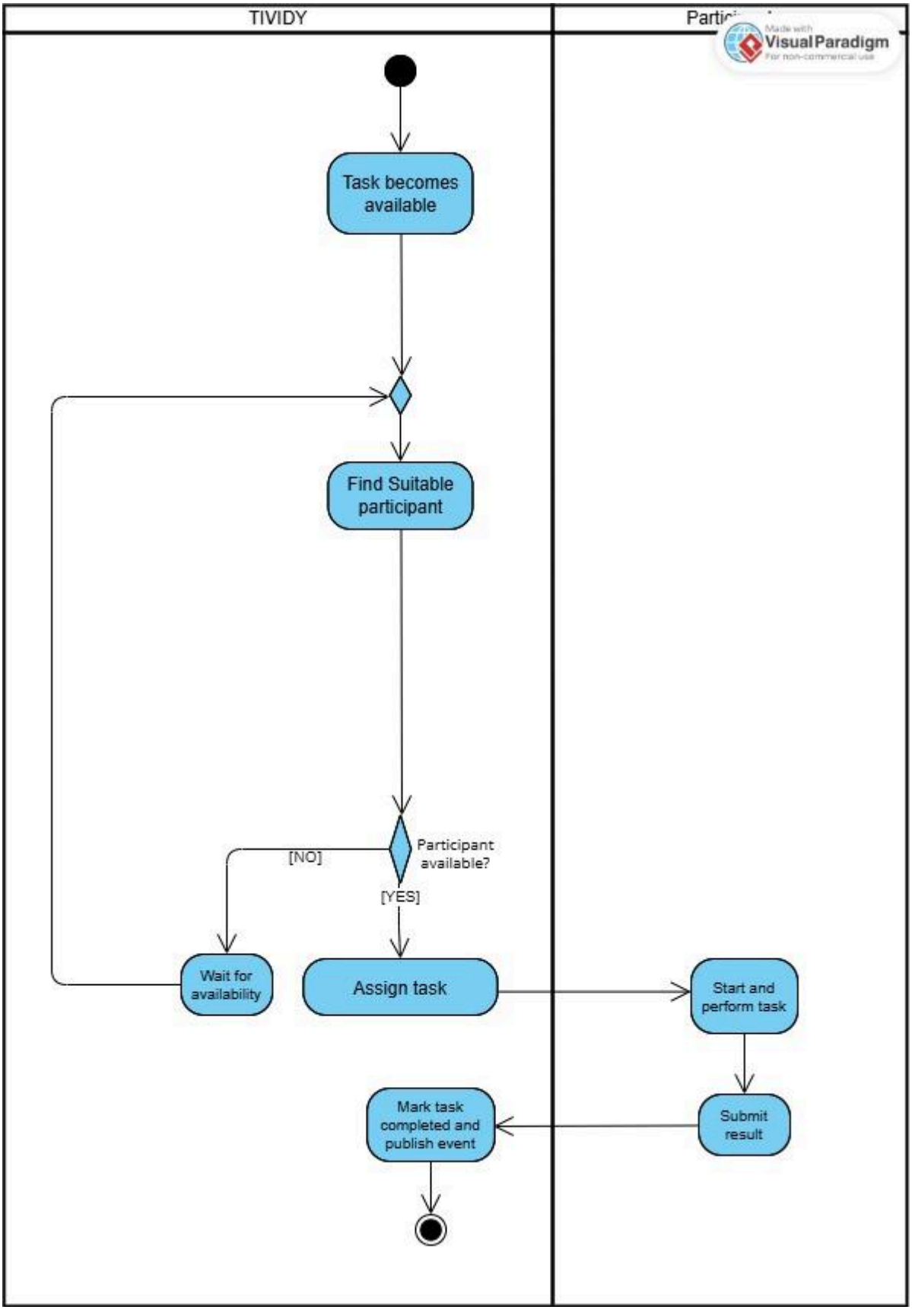
The expected outcome will be that a patient can successfully be admitted or re-admitted for a bed. Referred, waitlisted, declined for failing pre-authorisation for a bed in a hospital.

Task 3

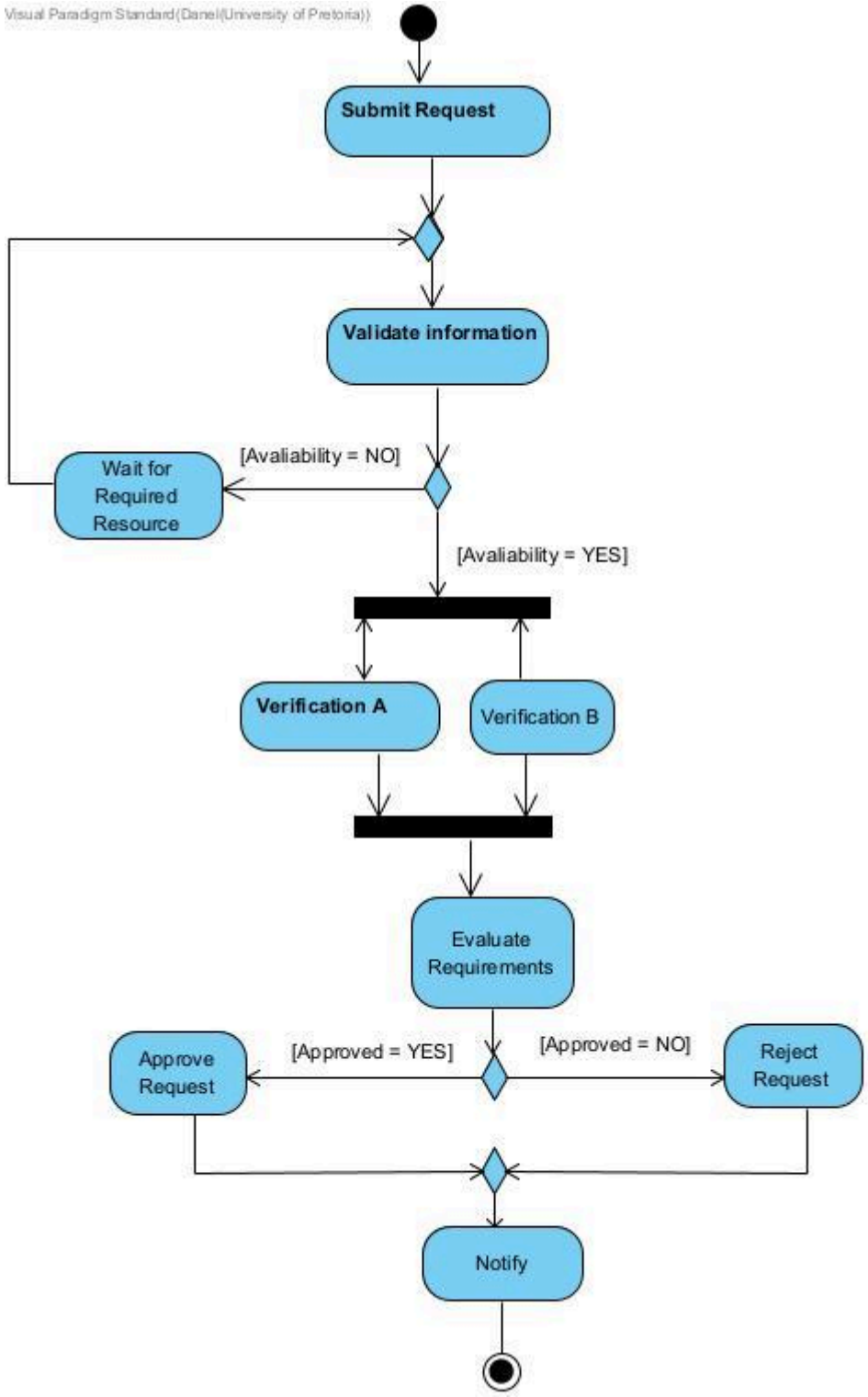
1. A generic workflow execution showing the progression of a workflow instance from creation or start until completion



2. Another Important generic TIVIDY workflow: Manual Task Assignment



3. Workflow from: Generic workflow



Task 4

1. State:
    a. Design problem: The valid functionality of a work item is dependent on the state it is currently in. This creates a problem, as a cancelled work item cannot be completed, and if an if/else chain is implemented, it makes the code less readable and causes every method to need updating to add a new state.
    b. Participants:
        i. State: WorkItemState.
        ii. ConcreteState: CreatedState, AvailableState, AssignedState. InProgressState, CompletedState, RejectedState, CancelledState, EscalatedState
        iii. Context: WorkItem.
    c. Collaborate: A WorkItem holds a reference to its current WorkItemState and forwards operations such as start() to it. The state either performs the operation or refuses it and sets the next state on the WorkItem when a transition is allowed.
    d. What varies: The functionality allowed for a work item changes at each state it could be in, as well as the actual available states. (The potential states can be different per process, per scenario, or per organisation).
    e. Why State: Without using State, every work item method would need an if/else chain on a status field to check whether it's valid. With State, each state becomes its own class that decides what each method does; the state class rejects invalid methods, and a new state is an entirely new class.

2. Decorator:
    a. Design problem: Different tasks need different additional functionality, such as validation, priority handling, auditing, notifications, or security, and often multiple at once. Creating a subclass for each combination would cause a large number of classes, and putting every feature in one task class would make it less readable and force unwanted functionality onto tasks that don't need it. The task must still be treated as the same kind of task by the workflow management system.
    b. Participants:
        i. Component: WorkComponent.
        ii. ConcreteComponent: ManualTask, AutomatedTask, ApprovalTask.
        iii. Decorator: WorkComponentDecorator.
        iv. ConcreteDecorator: ValidationDecorator, AuditDecorator, NotifyingItem, PriorityItem, SecureItem.

c. Collaborate: A decorator holds a WorkComponent, does its extra work before or after forwarding each call to the wrapped component, and implements the same interface so the rest of the system cannot tell the difference. Decorators are stacked when the definition is built.
d. What varies: The extra behaviours a task has, and in what combination. This can differ from task to task and from organisation to organisation.
e. Why Decorator: The Decorator pattern wraps the task in a decorator instead of changing the actual task. A decorator implements the same interface as the task it wraps, so the workflow management system treats wrapped and unwrapped tasks the same. Multiple decorators can wrap a task at runtime, so there is no need for a subclass for every combination, and adding new behaviour only requires one new decorator class, with no change to existing task classes.

3. Prototype:

a. Design problem: A change to one running instance should not accidentally change every other instance created from the same definition. Each instance created from a definition needs its own copy of the structure, data, and progress. Without the Prototype pattern, one way to do this is to rebuild the definition from scratch for every instance, which repeats the construction work and forces the creating code to know every concrete class.
b. Participants:
    i. Prototype: WorkComponent.
    ii. ConcretePrototype: WorkStage, ManualTask, AutomatedTask, ApprovalTask, WorkComponentDecorator, and its subclasses.
    iii. PrototypeManager: DefinitionRegistry.
    iv. Client: WorkflowManager.
c. Collaborate: DefinitionRegistry stores each finished WorkflowDefinition. When an instance is requested, WorkflowManager asks the registry for the definition and calls clone() on its base WorkComponent. Stage clones each of its children, and decorators clone the component they wrap, so the result is a full deep copy that becomes the new WorkflowInstance. Each applicant gets their own copy of the enrolment tree, so one applicant's progress never changes another's.
d. What varies: The workflow definition being cloned, which differs per process, and each instance's own data and progress once created. The creating code doesn't depend on the definition's concrete class.
e. Why Prototype: Builder constructs a workflow definition once. Prototype then clones it, as a deep copy of the whole Composite tree, to create each new instance. Because every stage and work item is copied and none are shared, edits to one instance never affect another or the original definition. The creating code only calls clone(), so it doesn't depend on the definition's concrete class.

4. Composite:
    a. Design problem: A workflow contains stages, which contain individual work items and sub-processes. If single work items and groups of work had different interfaces, every part of the system that handles work would need to check whether it is dealing with one item or a group. Ownership of the work must also be clear.
    b. Participants:
        i. Component: WorkComponent
        ii. Leaf: ManualTask, AutomatedTask, ApprovalTask
        iii. Composite: WorkStage
        iv. Client: WorkflowInstance
    c. Collaborate: Stage holds a list of WorkComponent children and passes requests like start() and cancel() on to them, in sequence or in parallel depending on its configuration. WorkflowInstance only calls the WorkComponent interface on the root. In enrolment, the root Stage "Enrolment" contains the Stages "Admission", "Fees" and "Registration", and each contains its own work items.
    d. What varies: The tree structure of the work, meaning how deeply it nests, which items are grouped into which stages or sub-processes, and whether a group runs sequentially or in parallel, differs per process.
    e. Why Composite: Single work items (leaves) and groups (composites) implement the same interface, so that the system can start, complete, or query one task or a whole sub-process in the same way, and a group passes the request on to its children. A group owns its children, which makes ownership and lifetime clear. A new kind of stage or work item can be added without changing the code that uses the interface.

5. Iterator:
    a. Design problem: Parts of the system need to go through the work in a workflow, for example, to find the next available item or to show progress, but the work is stored as a nested tree structure. If that code accessed each group's internal storage, it would depend on how groups store their children and would break whenever the storage changed.
    b. Participants:
        i. Iterator: WorkIterator
        ii. ConcreteIterator: DepthFirstIterator, ReadyItemIterator
        iii. Aggregate: WorkStage
        iv. ConcreteAggregate: WorkComponent
    c. Collaborate: A client calls createIterator() on a WorkComponent and steps through the result with hasNext() and next(), without seeing how Stage stores its children. WorkflowInstance uses ReadyItemIterator to find items that can become available, and the monitoring code uses DepthFirstIterator to walk the whole tree.
    d. What varies: The traversal order (for example, sequential or depth-first through nested sub-processes) and the internal storage of the groups.

e. Why Iterator: An iterator gives the client a standard way to traverse through work without knowing how a group stores its children. Each traversal is its own iterator class, so a new way to traverse the structure changes neither the Composite nor its client. Several iterators can be active at once, each keeping its own position.

6. Strategy:

a. Design problem: Different organisations decide who gets the next task in different ways, ie by role, by current workload, or by a rule. If this logic were written inside the work item classes, those classes would need rewriting, or a conditional chain, for every organisation.
b. Participants:
    i. Strategy: AssignmentStrategy
    ii. ConcreteStrategy: RoleBasedAssignment, WorkloadAssignment, RuleBasedAssignment
    iii. Context: TaskRouter
c. Collaborate: When a WorkItem becomes available, TaskRouter calls selectParticipant(item, candidates) on the AssignmentStrategy set for that item in the definition, and assigns the item to the result. TaskRouter never checks which concrete strategy it holds. In enrolment, "assess application" could use RoleBasedAssignment, while a busy period could switch to WorkloadAssignment.
d. What varies: The rule for how the program chooses who receives the next task. Which rule applies can differ per process or organisation.
e. Why Strategy: Each rule is its own class behind a common interface, so the part of the system that assigns work calls the rule without knowing which one it is. A new rule is a new class, allowing the work item classes to stay unchanged. The rule can be chosen per workflow definition, and no if/else chain for the type of organisation is needed.

7. Chain of Responsibility:

a. Design problem: Approval may involve one person, several levels, or escalation when nobody responds. If one approval class held all of these cases, it would contain a conditional chain for each case and need editing whenever an approval route changed.
b. Participants:
    i. Handler: ApprovalHandler
    ii. ConcreteHandler: RoleApprovalHandler, FinalApprovalHandler
    iii. Client: ApprovalRequest
c. Collaborate: ApprovalTask creates an ApprovalRequest and sends it to the first handler. Each RoleApprovalHandler either handles the request or passes it to its successor, and a missed deadline (reported by DeadlineEscalator from Observer) also passes it on. FinalApprovalHandler handles anything that reaches the end of the chain, so no request is left unhandled. In enrolment,

the chain is faculty reviewer, then head of department, then dean, all built from RoleApprovalHandler.

d. What varies: The number of approval levels, who handles each level, and the conditions under which a request is passed up (for example, rejection, no response by a deadline, or a value above a threshold).
e. Why Chain of Responsibility: Each approver or level is a handler that either deals with the request or passes it to the next handler. A route with one approver or five is just a chain of varied length. Escalation means passing the request to the next link, and the requester does not need to know who ends up handling it. Changing a route means rearranging handlers, not rewriting logic. The chain needs a final handler so that a request is never left unhandled.

8. Observer:

a. Design problem: A workflow system must react to what happens around it. Completing one activity may make another available, an external event may trigger new work, and several parts of the system may need to react to the same event. If the work item called each of these directly, it would depend on all of them and need editing whenever another was added.
b. Participants:
    i. Subject: EventPublisher
    ii. ConcreteSubject: WorkflowEventBus
    iii. Observer: EventListener
    iv. ConcreteObserver: DependencyActivator, AuditLogger, NotificationSender, MonitoringCollector, DeadlineEscalator
c. Collaborate: Listeners register with EventPublisher for the event types they care about. When a work item completes, or an external event arrives (for example, "payment received"), the publisher notifies every registered listener. DependencyActivator asks ResourceCoordinator whether dependent items can now become available, AuditLogger records the event, and NotificationSender notifies the applicant.
d. What varies: The set of interested parties for an event, which can grow or differ per process, and what each one does in response.
e. Why Observer: The subject announces an event to its registered observers without knowing what they are or what they do. A new interested party is a new observer registered with the subject, with no change to the subject itself. This keeps reactions separate from the core workflow model.

9. Command:

a. Design problem: A workflow system must be able to explain what happened. Actions such as assign, approve, reject, and escalate, along with the state changes they cause, must be retained as history and undone if needed. If actions were only method calls, nothing would record them or know how to reverse them.
b. Participants:

i. Command: WorkflowCommand
ii. ConcreteCommand: AssignCommand, CompleteCommand, ApproveCommand, RejectCommand, CancelCommand, StartCommand, EscalatedCommand, MakeAvailableCommand
iii. Client: WorkflowManager
iv. Invoker: CommandProcessor
v. Receiver: WorkItem

c. Collaborate: WorkflowManager creates a command for the requested action and passes it to CommandProcessor, which executes it on the receiving WorkItem and pushes it onto its history. Calling undo() pops the latest command and reverses it where applicable. In enrolment, undoing RegisterModules or a rejected approval is possible, but undoing "send confirmation message" is not, so that command's undo() refuses.
d. What varies: The actions performed on work, and what undoing each one means. Some actions may not be undoable.
e. Why Command: Each action becomes an object that holds the details of what it does, so it can be stored in a history, queued, or undone. The invoker does not need to know which concrete action it runs. A new action is a new command class. Command can also use Memento to hold the state needed for undo.

10. Memento:

a. Design problem: Rollback and recovery require restoring an earlier state of a workflow instance, for example after a failure. The instance's internal state should not be exposed to the rest of the system just so it can be saved.
b. Participants:
i. Originator: WorkflowInstance
ii. Memento: InstanceSnapshot
iii. Caretaker: SnapshotHistory
c. Collaborate: Before a risky step, CommandProcessor asks the WorkflowInstance to createSnapshot(), which copies the state it needs into an InstanceSnapshot, and SnapshotHistory stores it without reading it. If the step fails, the instance is given the snapshot back and restores itself. In enrolment, a snapshot is taken before registration so a failed payment can roll the application back.
d. What varies: Which state is captured, how much of it, and when (for example, before a risky step or at each state change).
e. Why Memento: The originator creates a snapshot of its own state, and only the originator can read it back, while the caretaker stores snapshots without seeing inside them. This allows rollback without breaking encapsulation. Snapshots cost memory and time, so they should be taken only when necessary.

11. Template Method:

a. Design problem: Every workflow instance runs through the same overall execution steps (for example, start, make work available, perform, check the outcome, then complete or handle failure), but the details of some steps differ per process. Copying the skeleton for each process would duplicate the common logic and let the sequences drift apart.
b. Participants:
    i. AbstractClass: WorkItem
    ii. ConcreteClass: ManualTask, AutomatedTask, ApprovalTask
c. Collaborate: execute() calls a fixed sequence of steps (check that the item is ready, onEnter(), perform(), recordResult()). The subclasses override perform(), so a manual task waits for a person, an automated task calls a Connector, and an approval task sends a request into the chain. Decorators wrap execute() from outside, so the two patterns don't duplicate each other.
d. What varies: The steps that differ per process, inside an execution order that stays fixed.
e. Why Template Method: The base class defines the fixed execution skeleton once, and subclasses override only the steps that vary. The order and shared logic live in one place, so every process runs the same way. Note: This pattern has no direct line in the brief, so it must have real varying steps in the final design.

12. Facade:

a. Design problem: TIVIDY has many subsystems (definitions, instances, routing, events, history, and reporting). If clients called each one directly, they would depend on the internal structure and need to know the correct order of calls.
b. Participants:
    i. Facade: WorkflowManager
    ii. SubsystemClasses: WorkflowInstance, TaskRouter, EventPublisher, CommandProcessor, SnapshotHistory
c. Collaborate: A client calls operations like createInstance(definitionName), start(instanceId), complete(itemId), and getStatus(instanceId) on WorkflowManager. The manager coordinates the subsystems, such as cloning the definition, wiring the listeners, starting the first items, and running commands, so the client needs no knowledge of the order. The enrolment scenario only talks to WorkflowManager.
d. What varies: The subsystems behind the entry point, which can change without clients changing.
e. Why Facade: One class offers simple operations such as create, start, and query a workflow, and handles the coordination between subsystems. Clients depend on that one interface and not on the engine's internals. Subsystems can still be used directly where needed.

13. Mediator:

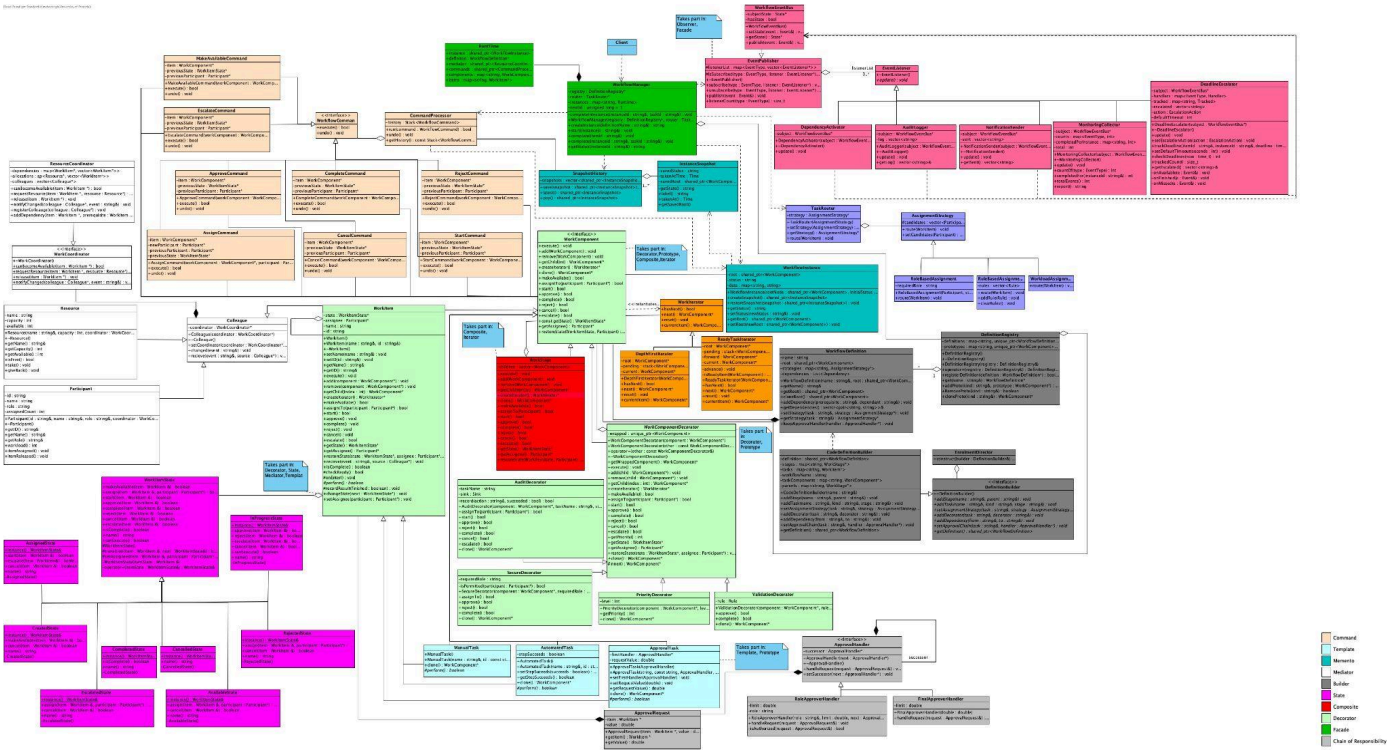
a. Design problem: Work items, participants, and resources depend on each other for example, one task waits for another, or participants share a resource. If they referenced each other directly, each class would need to know about the others, and adding a dependency would mean editing many classes.
b. Participants:
    i. Mediator: WorkCoordinator
    ii. ConcreteMediator: ResourceCoordinator
    iii. Colleague: Colleague
    iv. ConcreteColleague: WorkItem, Participant, Resource
c. Collaborate: Work items, participants, and shared resources hold a reference to WorkCoordinator and never to each other. When an item is about to become available, DependencyActivator asks the coordinator whether its prerequisites are complete and the resources it needs are free, and ResourceCoordinator answers from its own records. In enrolment, "register for module" can only start once the fee is paid and the module has a free place.
d. What varies: The dependencies and interaction rules between work items, participants, and resources.
e. Why Mediator: A mediator holds the coordination logic, and the colleagues talk only to it instead of to each other. Changing how they interact means changing the mediator and not every colleague.

14. Builder:

a. Design problem: A workflow definition is a complex nested structure of stages, tasks, rules, and dependencies. Constructing it with one large constructor, or by hand in client code, is error-prone, and the same construction process may need to produce different definitions.
b. Participants:
    i. Builder: DefinitionBuilder
    ii. ConcreteBuilder: CodeDefinitionBuilder
    iii. Director: EnrolmentDirector
    iv. Product: WorkflowDefinition
c. Collaborate: The director calls the builder's steps in order (addStage, addTask, setAssignmentStrategy, addDecorator, addDependency, setApprovalChain) and finally calls getDefinition(). The finished WorkflowDefinition is stored in DefinitionRegistry, where Prototype clones it. All enrolment-specific knowledge is in EnrolmentDirector, not in the Workflow management system.
d. What varies: The contents of the definition (stages, tasks, rules, and dependencies) and the source it is built from.
e. Why Builder: The builder assembles the definition step by step and returns the finished product, keeping construction separate from the structure itself.

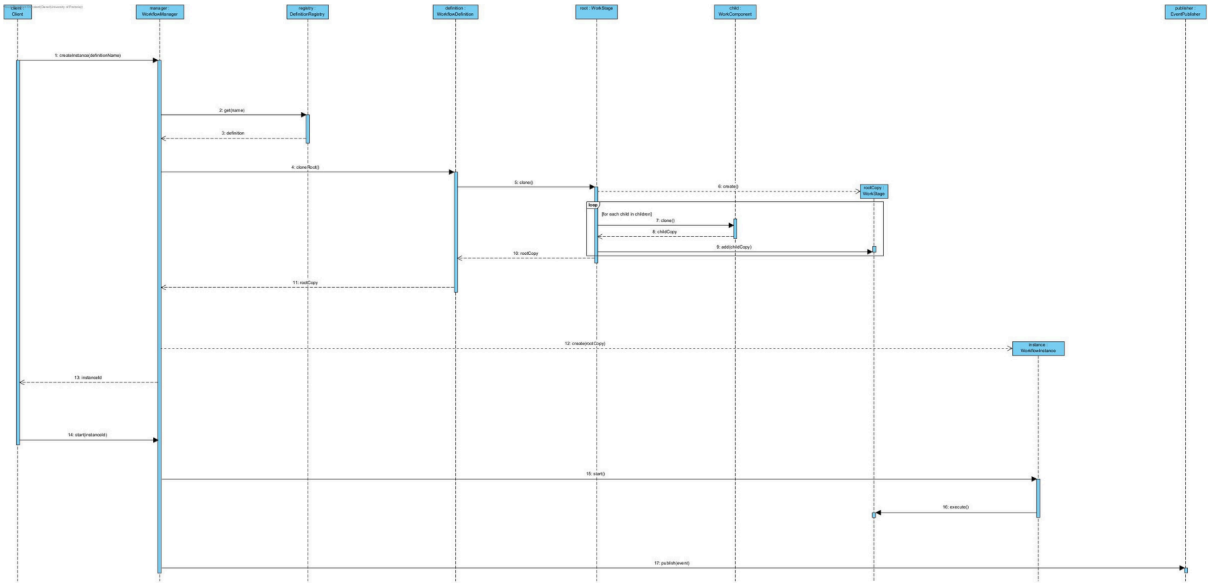
The finished definition is then cloned by Prototype to create instances. The Director is only needed if there is a reusable construction sequence.

Task 5

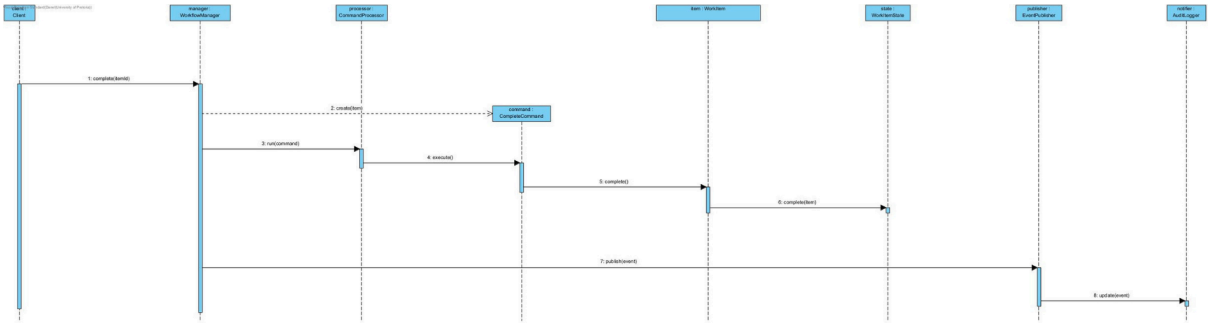

Command
Template
Memento
Builder
State
Composite
Decorator
Facade
Chain of Responsibility

Task 6

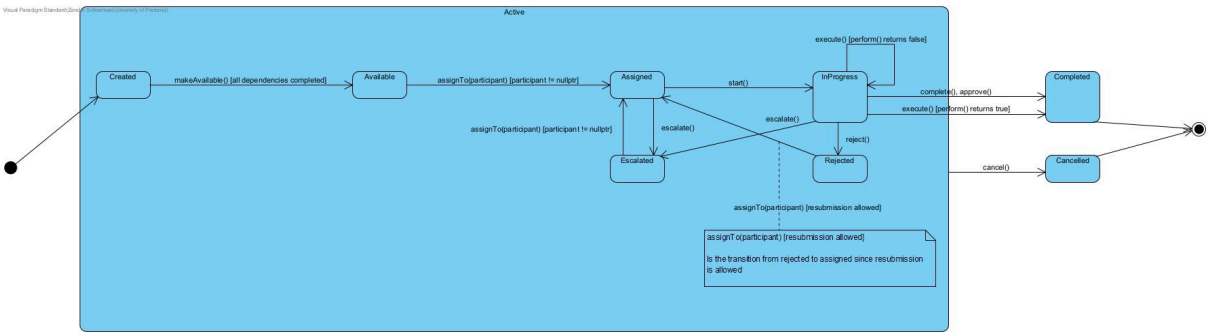
SD Diagram 1: Creating and Starting an instance.



SD Diagram 2: Complete work item



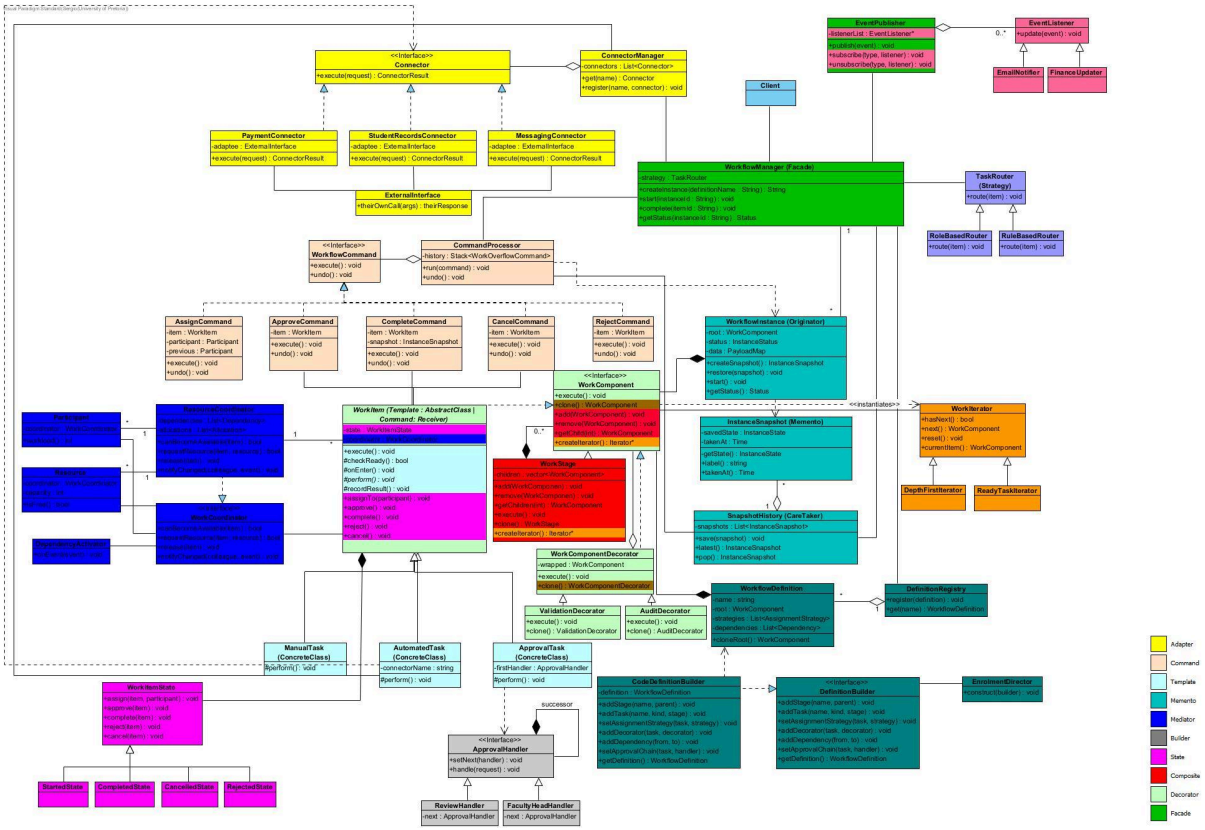
STD Diagram 1: Work Item Lifecycle



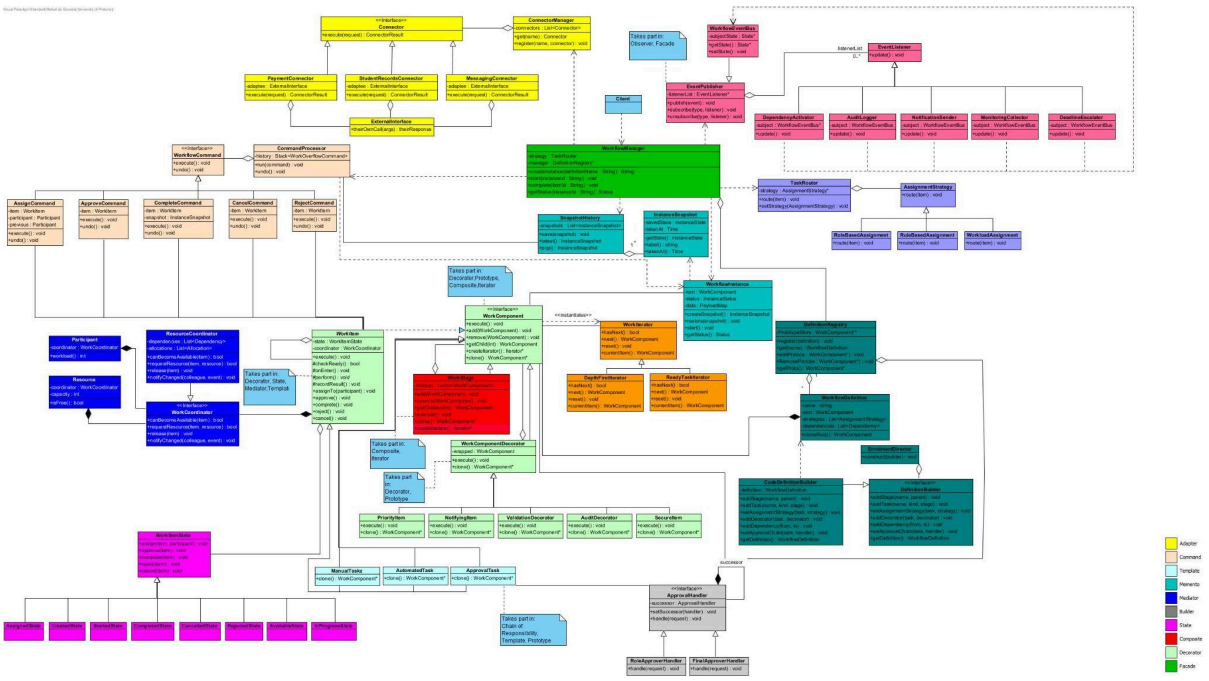
Task 7

Initial Class Diagram & Changes made:

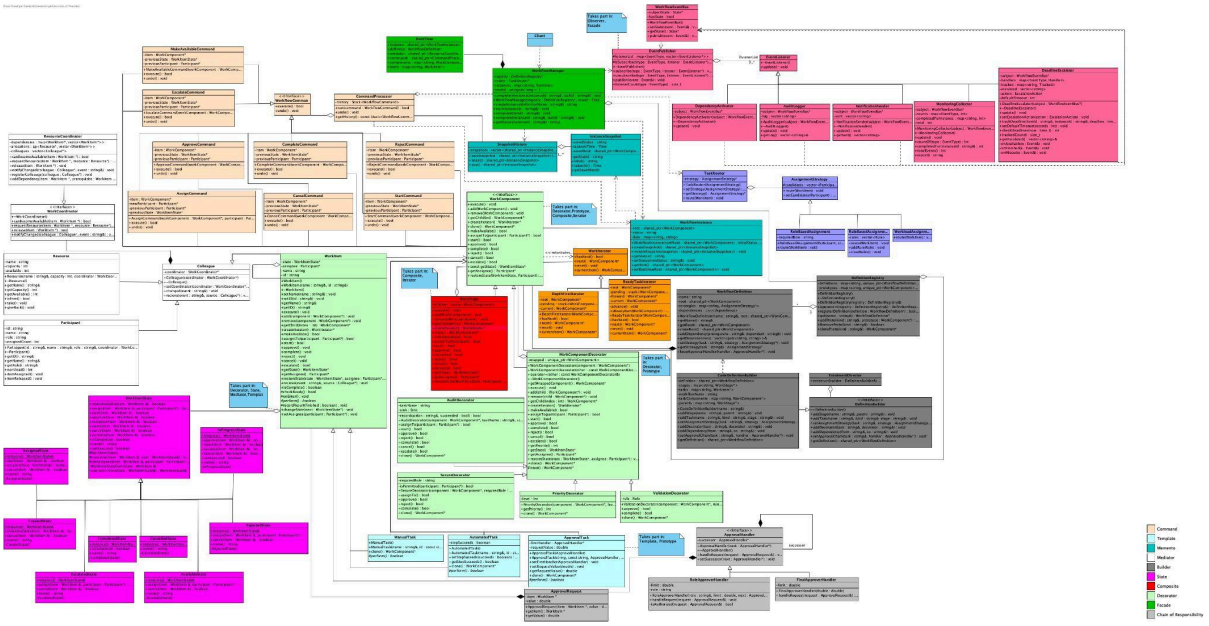
This was the initial class diagram from practical 6.
Major changes were made when compared to our final diagram updated in Task 5:
Adapter has been entirely removed as it was decided to be unnecessary and complicated to implement for a generic workflow.
Updates were made to each pattern's UML to stay consistent with our implementation.



This was our second iteration of our class diagram, which fixed all of the color issues the first one had, added some missing classes, as well as some of our relationship lines crossing each other and going over classes. This was before we completed the implementation fully, so there were many missing attributes and methods from the majority of the classes.

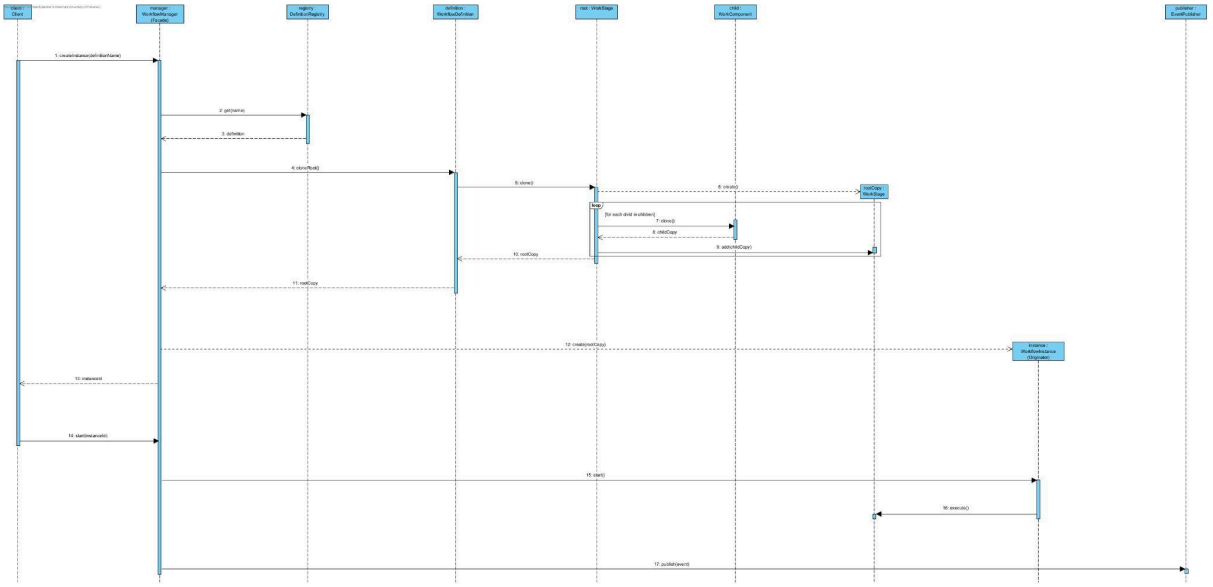


For our third and final iteration of the class diagram, we decided to wait and fix other issues before finalizing changes anymore so that everyone could be on the same page. In this iteration, we added the rest of the missing classes that were in task 4 and not in the diagram, we also fixed the rest of our relationship lines crossing each other and going through classes, we removed the adapter pattern, and finally, we updated the methods to match the .h files from the implementation.

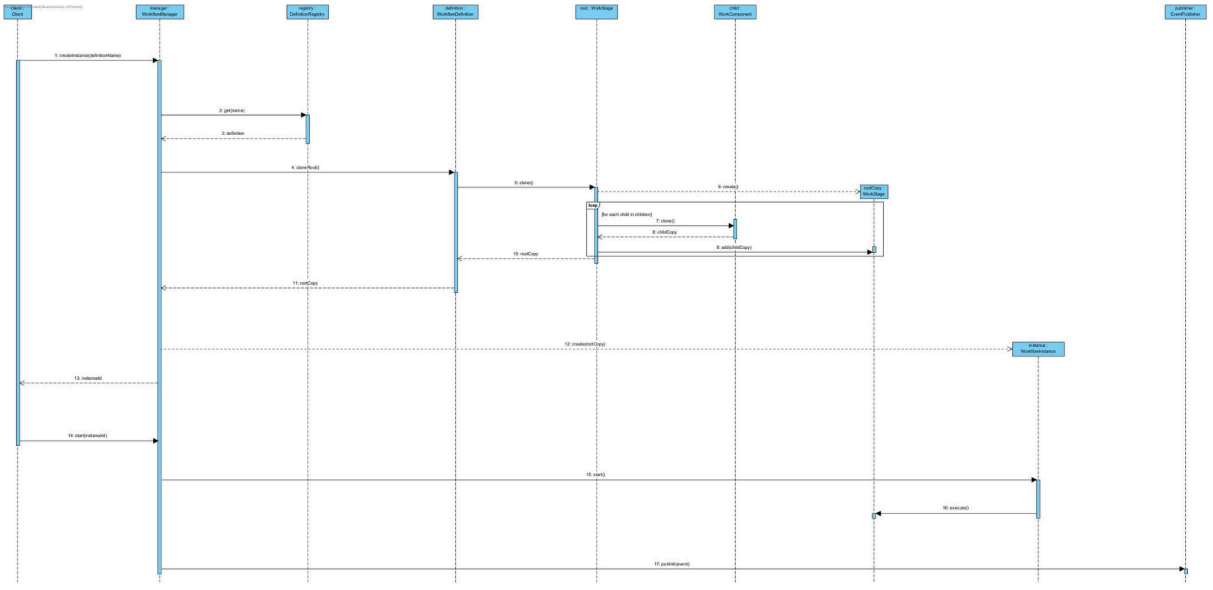


Sequence diagram changes:

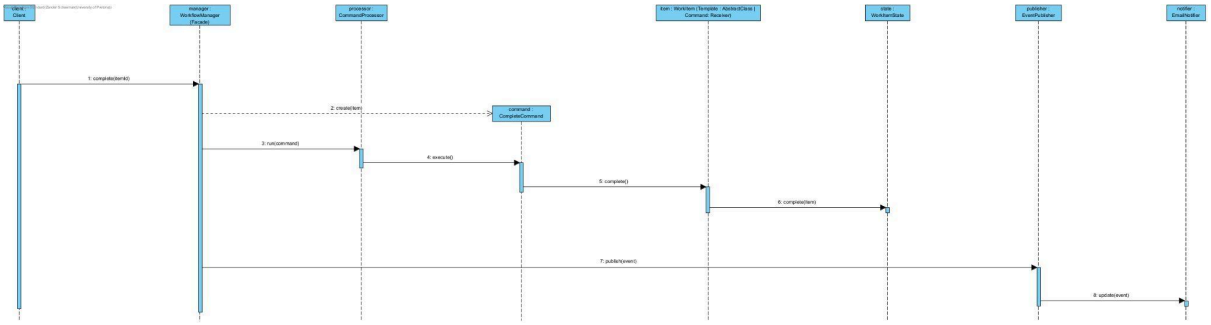
This was the first iteration of our Create and Start instance sequence diagram. This diagram had naming issues due to the names being incorrectly done in our UML class diagram.



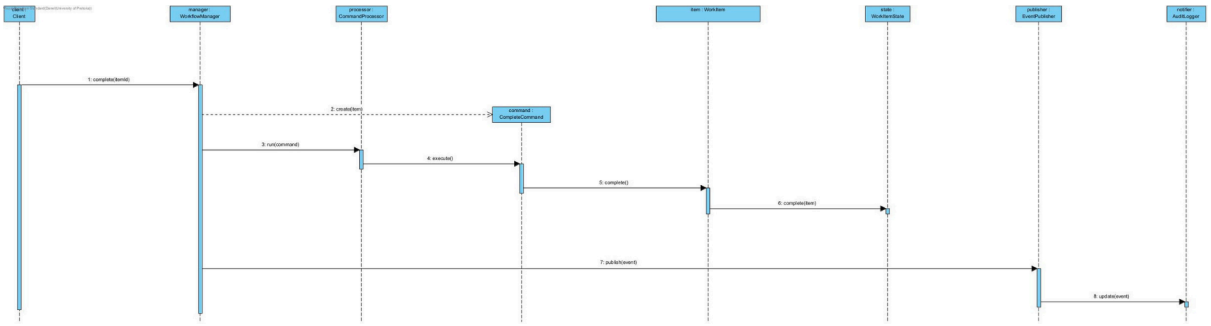
This was our second and final iteration. This diagram fixed the naming issues caused by the incorrect UML class diagram.



This was the first iteration of our Complete WorkItem sequence diagram. This diagram had naming issues due to the names being incorrectly done in our UML class diagram.

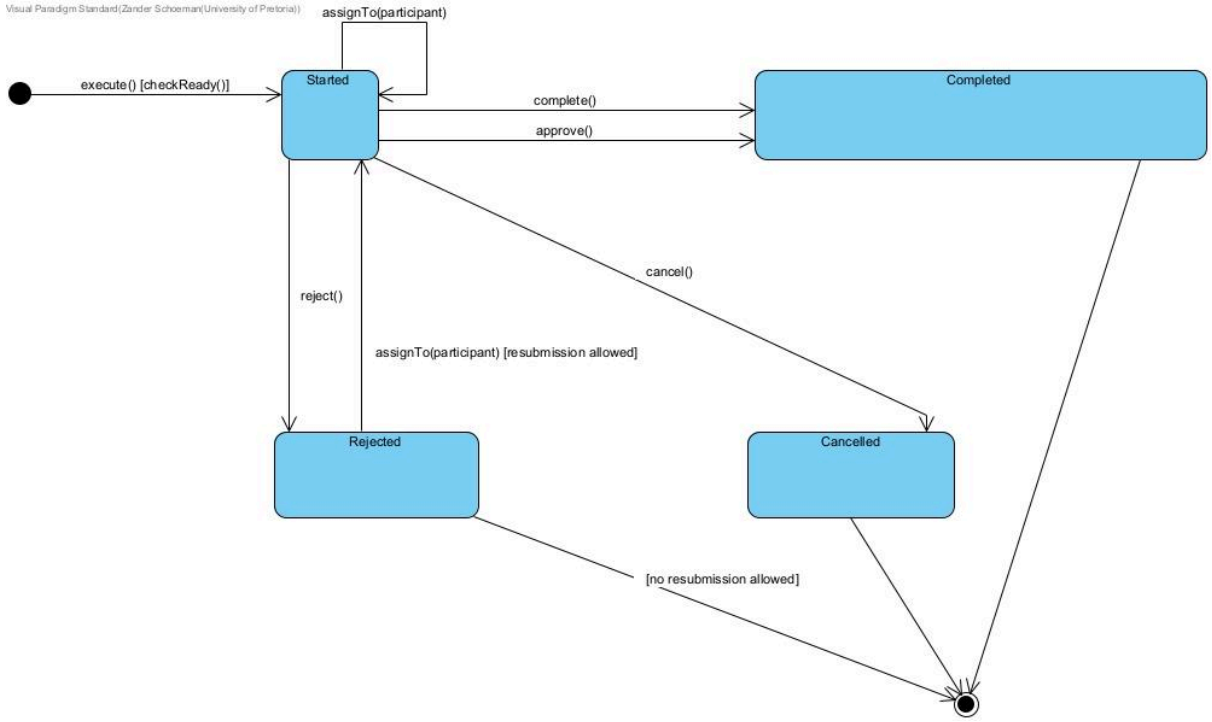


This was our second and final iteration. This diagram fixed the naming issues caused by the incorrect UML class diagram.

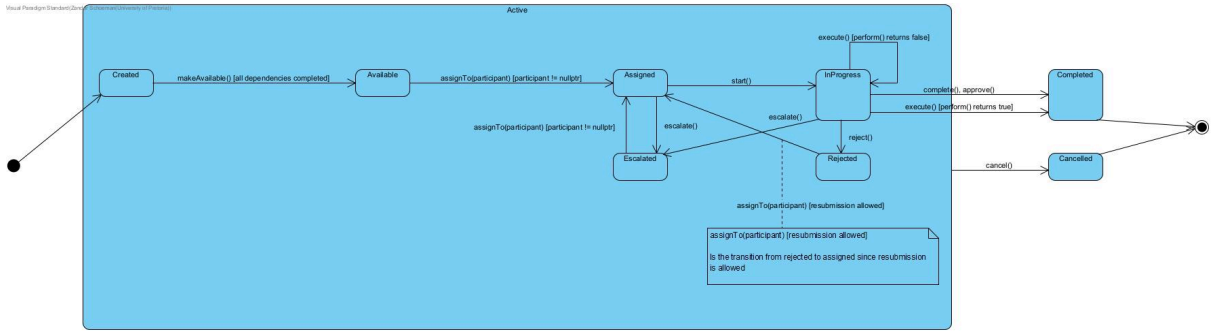


State diagram changes:

This was the first iteration of our WorkItem life cycle diagram, which was incorrect because not all the states were in the first UML class diagram and the adapter was being removed.

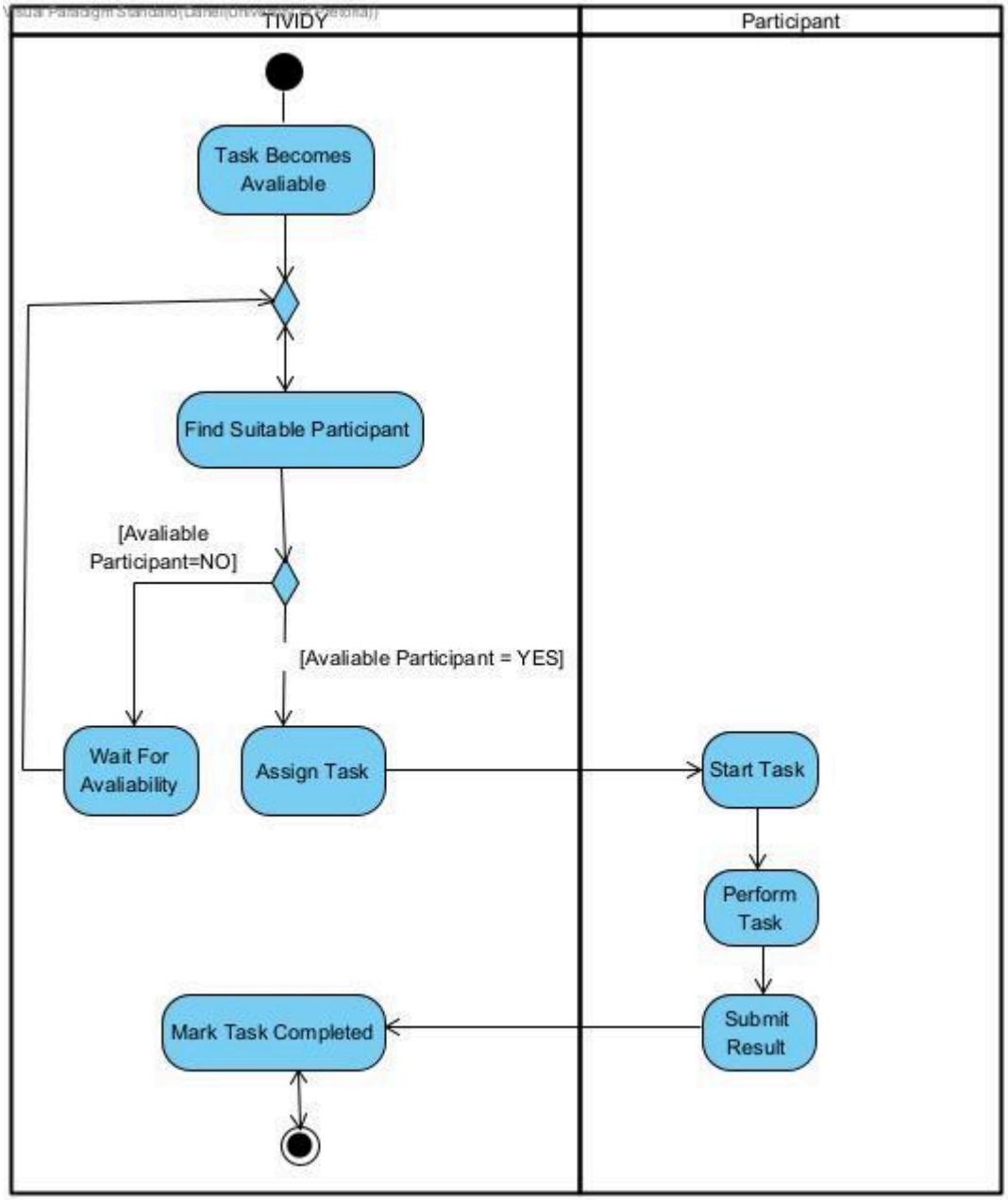


This was the second and final iteration of our WorkItem life cycle diagram. Since we added multiple additional states, the entire diagram had to be reworked to account for the new states.

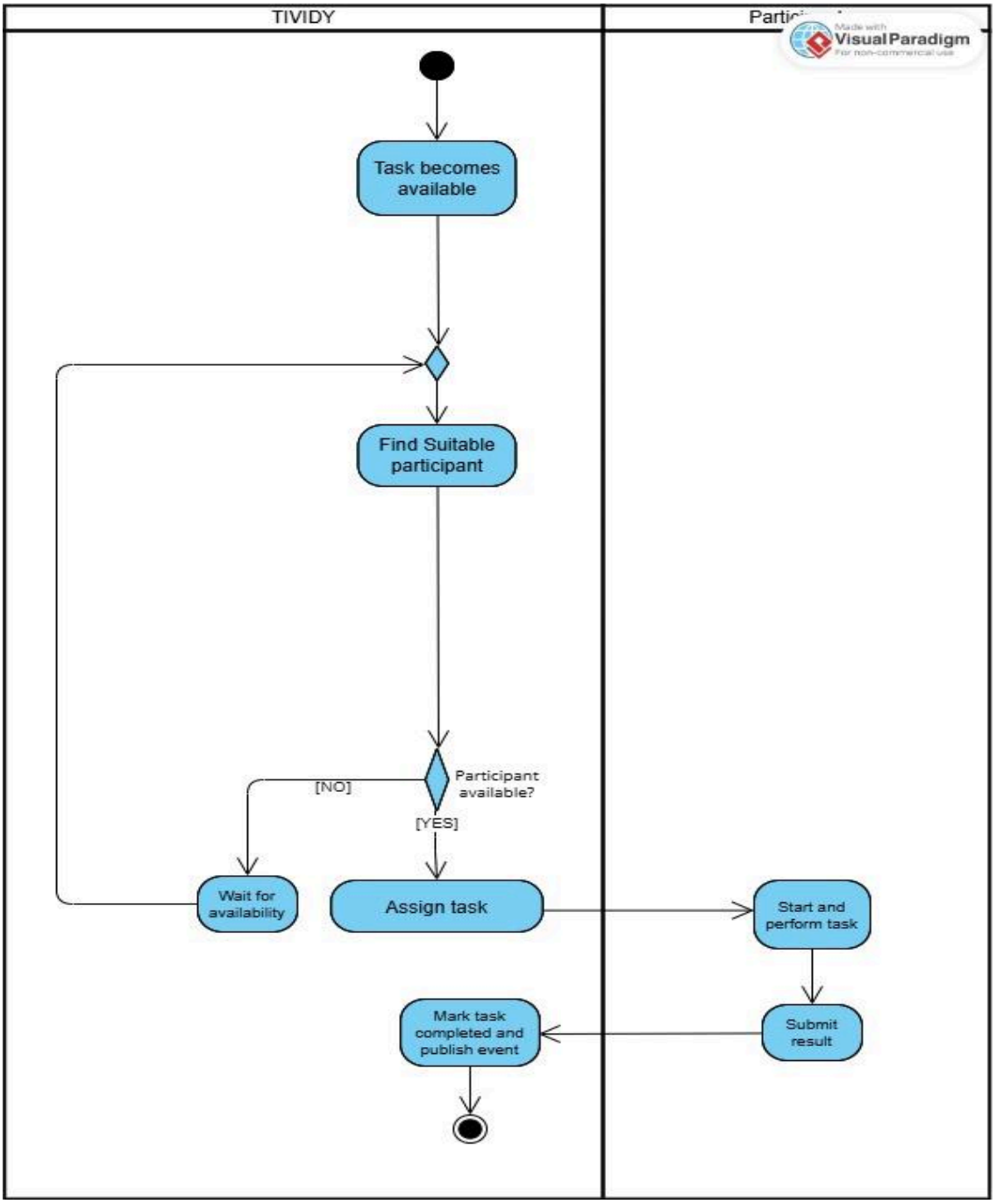


Activity diagram changes:

This was the first iteration of our Manual Task Activity diagram. This diagram had syntax errors for the decision nodes, skew lines, and activities split into multiple actions.



This was the second and final iteration of our Manual Task Activity diagram. It fixed syntax errors and merged action nodes that were incorrectly splitting activities.



Contributions:

Rafael de Gouveia - u25054903:

For this project, I handled task 4 of practical 6, which involved choosing the patterns we used based on the specifications given and how the patterns worked together, as well as documenting each pattern with the problem it solved, the participants, the way the pattern collaborated with other patterns, what might change in the patterns from generic to scenario, and why that pattern specifically was chosen over other patterns. For the project section, I helped with all the class diagram iterations as we found they needed to change, and I coded the implementation and Doxygen comments for the Mediator and the Chain of Responsibility patterns. I also handled the State, Strategy, Chain of Responsibility, Observer, Mediator, Command, and Memento patterns for the presentation slides and generated the QR code for them. Finally, I assisted in documenting the changes made to add to task 7 and updated task 3 and 6 diagrams in the documents.

Logan van der Merwe - u25025822

For this project, I tackled Task 2 of practical 6. In this task, I planned and described a specific workflow (i.e., a university enrolment workflow), then converted it into a generic workflow using the same building blocks.

In addition to laying the groundwork for the generic workflow system, I implemented the Observer and Strategy patterns, wrote the README, and provided helpful guides and documents detailing subtasks of larger documents, running instructions, et cetera. While doing this I organised the docs folder of the GitHub as well. When we decided to remove the Adapter pattern, I also helped confirm which diagrams were affected by this change and what needed to be adjusted, if needed. I contributed to the slideshow and also helped with the Docker testing.

Elme Hattingh - u25045505

For this project, I took on the role of team leader, which entailed delegating patterns to everyone in a fair manner and also helping resolve conflict, as well as doing the research on workflows, which was Task 1 of Prac 6. Furthermore, I implemented the code for the Decorator and Prototype patterns, as well as creating the Doxyfile from the Doxygen comments and fixing the errors that came up in the warnings log. I also built the Dockerfile and its supporting files in order to do the testing, and fixed the errors that appeared. I also contributed to the README.

Hanru Jansen van Rensburg - u25407725

For this project, I helped with Task 5, which is the class diagram. This includes the initial creation of the diagram and continuous updates to it throughout the whole project as changes were made to our implementation. I implemented the Memento pattern as well as the Command pattern and handled changes to be made to those patterns to accommodate minor bugs. I contributed to the slides, specifically the GitHub, Docker & CI/CD sections, to showcase how we used Git/GitHub, etc.
Also helped with Task 7, which had to do with changes to our diagrams throughout Prac 6 and the Projects.

Danel Steyn u24943802

For Practical 6, I did Task 3, creating the 3 different workflow diagrams to explain the system's processes and execution flow. I was also responsible for implementing the Builder and Facade design patterns. This also entailed some integration with the patterns involved from other teammates. I also took leadership in planning and designing the presentation, ensuring that the content was well-structured, visually consistent, and appealing.

Sergio Ferreira u25247418

For practical 6, I did Task 5 alongside Hanru and Rafael, naming all the initial patterns that were to be used for the project, creating the initial class diagram, and providing continuous updates to said diagram throughout the duration of the project. I was also responsible for the implementation of the Composite and Iterator patterns, alongside bug fixes to any other classes involving said patterns. I also handled creating the presentation slides for the Facade, Builder, Prototype, Composite, Iterator, Decorator, and Template Method patterns.

Zander Schoeman - 25097726

For practical 6, I did Task 6, which included doing two sequence diagrams and one state transition diagram, however, after we started changing functionality to accommodate the

requirements of the project, I did the new state transition diagram for the generic workflow.
I also implemented the State and Template Method Design patterns, as well as designing the web GUI, which is used to demonstrate the functionality of our project.